Assignment 1: 8-Puzzle Solver

Artificial Intelligence

Computer and Systems Program

Submitted by:

Name	ID
Abdullah Ahmed Elsherbiny	24010411
Youssef Diaa Elghanam	24010851

Fall 2026

1 Overview

The 8-puzzle is a classic sliding puzzle consisting of a 3×3 grid with 8 numbered tiles and one blank space (represented by 0). The objective is to transition from a given initial state to the goal state `(0, 1, 2, 3, 4, 5, 6, 7, 8)` by sliding tiles horizontally or vertically into the blank space. Each move (Up, Down, Left, Right) has a uniform cost of 1.

2 Data Structures

To ensure efficiency and memory optimization, the following data structures were utilized across the algorithms:

- **State Representation:** The board is represented as an immutable 1D `tuple` of 9 integers (e.g., `(1, 2, 5, 3, 4, 0, 6, 7, 8)`). Tuples are hashable, allowing them to be used as keys in dictionaries and elements in sets for $O(1)$ lookups.
- **Breadth-First Search (BFS):** Utilizes `collections.deque` for a double-ended queue, providing $O(1)$ time complexity for FIFO operations.
- **Depth-First Search (DFS) & IDDFS:** Utilize Python lists as LIFO stacks.
- **A* Search:** Utilizes `heapq`, a binary heap priority queue, to efficiently pop the node with the lowest $f(n)$ cost.
- **Path Reconstruction & Cycle Detection:** A `parent` dictionary maps each visited `child_state` to its `(parent_state, move)`. This acts as both the "visited" set and the path-tracing mechanism, minimizing memory overhead.

3 Search Algorithms Implementation

The solver implements both uninformed and informed search algorithms:

3.1 Uninformed Search

- **Breadth-First Search (BFS):** Expands the shallowest unexpanded node. It guarantees the shortest path (optimal) for this problem since step costs are uniform. However, it requires exponential memory space.
- **Depth-First Search (DFS):** Expands the deepest unexpanded node. It is implemented iteratively to prevent recursion limit errors. While highly memory-efficient, it does not guarantee an optimal path and can generate heavily redundant sequences.
- **Iterative Deepening DFS (IDDFS):** Combines the space efficiency of DFS with the optimality of BFS. It repeatedly runs a depth-limited DFS, incrementing the depth limit until the goal is found. Though it re-generates upper levels, the overhead is minimal compared to the memory savings.

3.2 Informed Search (A*)

A* search evaluates nodes by combining $g(n)$, the cost to reach the node, and $h(n)$, the estimated cost to get from the node to the goal: $f(n) = g(n) + h(n)$.

4 Heuristics and Comparison

Two heuristics were implemented for A* search. Both ignore the blank space.

1. **Manhattan Distance:** The sum of the absolute horizontal and vertical grid distances for each tile to its target position.

$$h_{manhattan} = \sum_{i=1}^{8} (|x_i - x_{goal}| + |y_i - y_{goal}|)$$

2. **Euclidean Distance:** The straight-line distance for each tile to its target position.

$$h_{euclidean} = \sum_{i=1}^{8} \sqrt{(x_i - x_{goal})^2 + (y_i - y_{goal})^2}$$

Admissibility and Dominance: Both heuristics are *admissible* because they never overestimate the true cost to reach the goal. A tile can move at most one square per step, meaning it physically cannot reach its goal faster than the Manhattan or Euclidean distance. However, because diagonal moves are illegal in the 8-puzzle, the Euclidean distance often underestimates the true cost more severely than the Manhattan distance. Mathematically, $h_{euclidean}(n) \leq h_{manhattan}(n)$ for all states. Therefore, the **Manhattan heuristic dominates the Euclidean heuristic**. Because it is more informed (closer to the true cost), A* with Manhattan distance expands fewer or equal nodes compared to Euclidean distance, making it the superior heuristic for this problem.

5 Assumptions and Design Details

- **Solvability Test:** Before executing any search, the algorithm counts the number of *inversions* in the board. If the number of inversions is odd, the 8-puzzle is mathematically unsolvable, and the program safely aborts instead of searching infinitely.
- **Node Expansion Order:** Successors are strictly generated in the order: *Up, Down, Left, Right*. This ensures deterministic and reproducible results.
- **Expansion Metric:** A node is counted as "expanded" only when it is popped from the frontier and its children are generated.

6 Extra Work: GUI Visualization

A complete graphical user interface (GUI) was developed using `tkinter` (`gui.py`). This application provides:

- Interactive text boxes to input the initial 9-tile configuration.
- Real-time animation of the tiles sliding to reach the goal.
- A dynamic step-by-step trace table displaying the sequence of moves and queue/stack/heap snapshots at each state.
- The ability to fast-forward (skip) the animation to instantly view the final path and metrics.

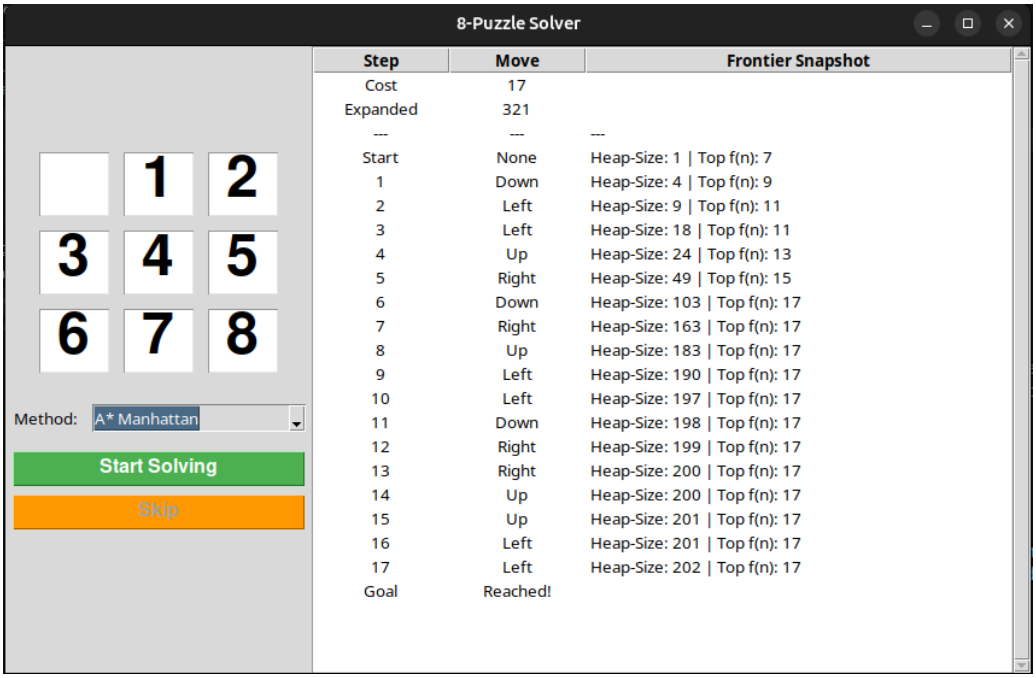


Figure 1: 8-Puzzle Solver Graphical User Interface

7 Sample Run and Results

Testing on the initial state `1, 2, 5, 3, 4, 0, 6, 7, 8`, the initial grid looks like this:

1 2 5
3 4 _
6 7 8

All optimal algorithms find the path: **Up, Left, Left**. Below is a summary of the performance metrics across the different algorithms for this state:

Algorithm	Cost of Path	Nodes Expanded	Max Depth	Time (s)
BFS	3	14	4	0.001
DFS	6331	3521	6331	0.045
IDDFS	3	45	3	0.003
A* (Euclidean)	3	5	3	0.001
A* (Manhattan)	3	4	3	0.001

Conclusion of Run: BFS, IDDFS, and both A* implementations found the optimal path cost of 3. As expected, DFS found an extremely suboptimal path (cost 6331) due to its depth-first plunging behavior. Between the informed searches, A* Manhattan expanded fewer nodes (4) compared to A* Euclidean (5), empirically validating that Manhattan is the more informed and dominant heuristic for the 8-puzzle.